

Bases de Datos

Clase 6y7: SQL

Lo que hemos visto

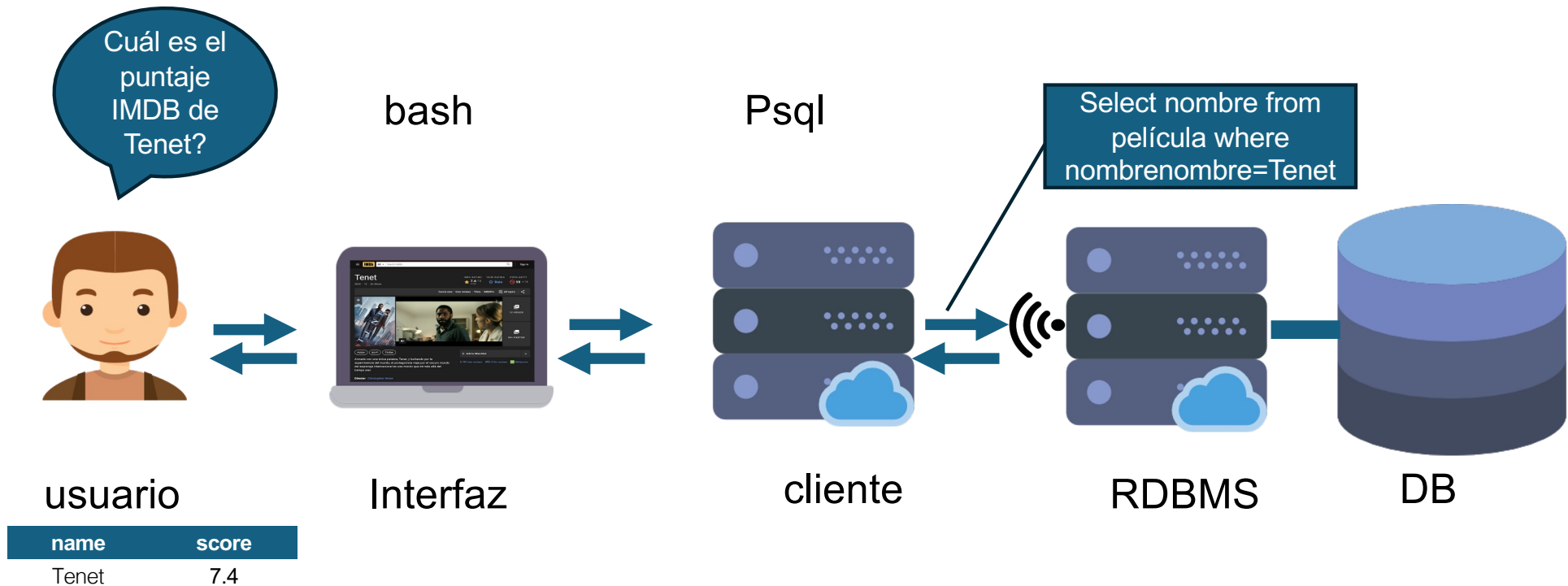
- Sabemos lo que es un modelo Relacional ✓
- Sabemos hacer consultas en Álgebra Relacional ✓
- Sabemos modelar un problema usando el modelo E/R ✓
- Sabemos transformar un MER a un esquema relacional ✓
- Dependencias Funcionales Anomalías ✓
- Formas Normales ✓

Ahora SQL

Recordemos como funciona un RDBMS

- Un DBMS relacional es un programa (**e.g. PostgreSQL**) que se instala en un computador (**servidor**)
- Este programa se mantiene escuchando conexiones (**Listener**)
- El usuario, generalmente otro programa cliente, (**e.g. PSQL**), se conecta al RDBMS y le puede entregar instrucciones.

DBMS Relacional



SQL

Structured Query Language

- Usado para todas las comunicaciones con bases de datos relacionales.
- Aplicaciones web, locales, móviles, análisis de datos, etc.
- Hasta algunas arquitecturas serverless lo requieren o usan [lenguajes basados en SQL](#).
- Software implementan “subconjunto” del estándar (cada uno tiene diferencias sutiles) [ISO/IEC 9075:2023](#)
- Lenguaje declarativo no procedural

Declarativo vs. Procedural

- SQL es declarativo, decimos lo que queremos, pero sin dar detalles de como lo computamos.
- El DBMS transforma la consulta SQL en en un algoritmo ejecutado sobre un lenguaje procedural
- Un lenguaje como Python es procedural: para hacer algo debemos indicar paso a paso el procedimiento

En simple: Lo que quiero vs cómo lo obtengo

Structured Query Language sublenguajes

- DDL: Lenguaje de definición de datos
 - Crear y modificar tablas, atributos y llaves.
- DML (Data Manipulation Language / Lenguaje de Manipulación de Datos)
 - Se encarga de la manipulación de los datos almacenados en la base de datos.
- DCL (Data Control Language / Lenguaje de Control de Datos)
 - Gestiona los permisos y los derechos de acceso de los usuarios a los objetos de la base de datos.
- DQL (Data Query Language / Lenguaje de Consulta de Datos)
 - Permite realizar consultas y recuperar información de la base de datos.
- TCL (Transaction Control Language / Lenguaje de Control de Transacciones)
 - Controla las transacciones, asegurando la integridad de los datos.
- Lenguajes procedurales como PL/SQL, T-SQL, PL/pgSQL, etc.

Tipos de datos

- Caracteres (Strings)
 - `char(20)` - Largo fijo
 - `varchar(20)` - Largo variable
- Números
 - `int`, `smallint`, `float`, ...
- Tiempo y fecha
 - `time` - hora formato 24 hrs.
 - `date` - fecha
 - `timestamp` - fecha + hora
- ¡Y varios otros! Dependen del RDBMS que se esté usando.

Creando un esquema

Consideremos de ejemplo:

Peliculas(id, nombre, año, categoria, calificacion, director)

Actores(id, nombre, edad)

Actuo_en(id_actor, id_pelicula)

Sin datos

CREATE TABLE Peliculas(

id int,

nombre varchar(30),

año int,

categoria varchar(30),

calificacion float,

director varchar(30)

)

id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografia	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

CREATE TABLE Actores(

id int,

nombre varchar(30),

año int

)

id	nombre	año
1	Leonardo DiCaprio	1974
2	Matthew McConaughey	1969
3	Daniel Radcliffe	1989
4	Jessica Chastain	1977

CREATE TABLE Actuo_en(

id_actor int,

id_pelicula int,

)

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

Crear Tablas con llaves

Sin datos

```
CREATE TABLE Peliculas(
  id int PRIMARY KEY,
  nombre varchar(30),
  año int,
  categoria varchar(30),
  calificacion float,
  director varchar(30)
)
```

id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografia	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

```
CREATE TABLE Actores(
  id int PRIMARY KEY,
  nombre varchar(30),
  año int
)
```

id	nombre	año
1	Leonardo DiCaprio	1974
2	Matthew McConaughey	1969
3	Daniel Radcliffe	1989
4	Jessica Chastain	1977

```
CREATE TABLE Actuo_en(
  id_actor int,
  id_pelicula int,
  PRIMARY KEY (id_pelicula, id_actor)
)
```

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

Valores Default

Sintaxis general:

```
CREATE TABLE <Nombre> (...<atr> tipo DEFAULT <valor>...)
```

Ejemplo:

```
CREATE TABLE Peliculas(  
  id int PRIMARY KEY,  
  nombre varchar(30),  
  año int,  
  categoria varchar(30) DEFAULT 'Acción',  
  calificacion float DEFAULT 0,  
  director varchar(30)  
)
```

id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Modificar Tablas

Eliminar tabla:

DROP TABLE Peliculas

Eliminar atributo:

ALTER TABLE Peliculas **DROP COLUMN** director

Agregar atributo:

ALTER TABLE Peliculas **ADD COLUMN** productor varchar(30)

id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Insertar Datos

id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Sintaxis general:

```
INSERT INTO R(at_1, ..., at_n) VALUES (v_1, ..., v_n)
```

Ejemplo:

```
INSERT INTO Peliculas(id, nombre, año, categoria, calificacion, director) VALUES  
(321351, 'V for Vendetta', 2005, 'Action', 8.2, 'James McTeigue')
```

Ejemplo abreviado (asume orden de creación):

```
INSERT INTO Peliculas VALUES (321351, 'V for Vendetta', 2005, 'Action',  
8.2, 'James McTeigue')
```

Inserciones con llaves foráneas

¿Qué pasa en este caso?

```
CREATE TABLE R(a int, b int, PRIMARY KEY(a));
```

```
CREATE TABLE S(a int, c int, FOREIGN KEY(a) REFERENCES R, ...);
```

```
INSERT INTO R VALUES(1, 1);
```

```
INSERT INTO S VALUES(1, 2);
```

Todo bien hasta ahora...

Inserciones con llaves foráneas

¿Qué pasa en este caso?

```
CREATE TABLE R(a int, b int, PRIMARY KEY(a));
```

```
CREATE TABLE S(a int, c int, FOREIGN KEY(a) REFERENCES R, ...);
```

```
INSERT INTO R VALUES(1, 1);
```

```
INSERT INTO S VALUES(1, 2);
```

```
INSERT INTO S VALUES(2, 3);
```



ERROR!

La base de datos no permite que se agreguen filas en que la llave foránea no está en la tabla referenciada!

Consulta con SELECT

Forma básica

Las consultas en general se ven:

SELECT atributos

FROM relaciones

WHERE condiciones

Para ver todo de una tabla

SELECT * FROM Peliculas

id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Para ver nombre y calificación de todas las películas dirigidas por Nolan:

SELECT nombre, calificacion

FROM Peliculas

WHERE director = 'C. Nolan'

Para ver todo de una tabla con criterios

Para las películas estrenadas desde el 2010:

```
SELECT *  
FROM Peliculas  
WHERE año >= 2010
```

id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

WHERE permite =, <>, !=, >, <, <=, >=, AND, OR, NOT, IN, BETWEEN, etc...

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

WHERE permite =, <>, !=, >, <, <=, >=, AND, OR, NOT, IN, BETWEEN, etc...

Películas estrenadas entre 1971 y 1978:

```
SELECT *  
FROM Peliculas  
WHERE año BETWEEN 1971 AND  
1978
```

Películas estrenadas en 1971, 1973 y 2001:

```
SELECT *  
FROM Peliculas  
WHERE año IN (1971, 1973, 2001)
```

SELECT en General

La consulta:

```
SELECT a_1, ..., a_n  
FROM T_1, ..., T_m  
WHERE <condicion>
```

Se traduce al álgebra relacional como:

$$\pi_{a_1, \dots, a_n} (\sigma_{condiciones} (T_1 \times \dots \times T_m))$$

Para actualizar valores de una tabla:

UPDATE Peliculas

SET calificacion = 0

WHERE nombre = 'Sharknado 6'

id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Sharknado 6	2010	Terror	8.8	A. Ferrante



id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Sharknado 6	2010	Terror	0	A. Ferrante

Forma general

UPDATE R

SET <Nuevos valores>

WHERE <Condición sobre R>

<Nuevos valores> $\rightarrow$ (atributo₁ = nuevoValor₁, ..., atributo_n = nuevoValor_n)

Forma general

Para borrar filas que cumplan una condición:

DELETE FROM R

WHERE <Condición sobre R>

¿Qué pasa si se nos olvida el `WHERE` en un `UPDATE` o `DELETE FROM`?

`UPDATE Users`

`SET password = 'contraseña'`

`DELETE FROM Tweets`

... ¿O si se nos pasa un `;` entre medio?

`UPDATE Users`

`SET password = 'contraseña';`

`WHERE email = 'some@email.com'`

`DELETE FROM Tweets;`

`WHERE user_name = 'realDonaldTrump'`

Forma general

Para borrar filas que cumplan una condición:

DELETE FROM R

WHERE <Condición sobre R>

¿Qué pasa si se nos olvida el **WHERE** en un **UPDATE** o **DELETE FROM**?
... ¿O si se nos pasa un **‘;’** entre medio?

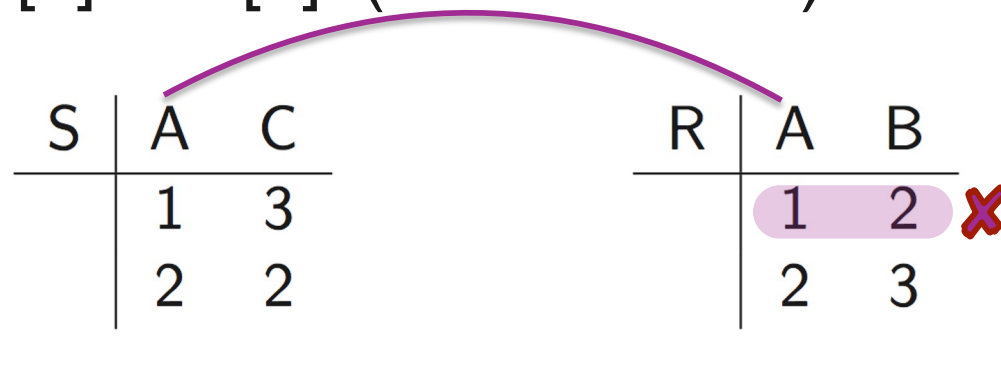
Se borran / actualizan todas las tuplas!!



Eliminar con llaves foráneas

Tenemos $S[a] \subseteq R[a]$ (llave foránea)

S	A	C	R	A	B
	1	3		1	2
	2	2		2	3



¿Qué ocurre al eliminar (1, 2) en **R**?

ERROR!

La base de datos no permite eliminar llaves foráneas usadas en otras tablas. **Pero hay alternativas**

Eliminar con llaves foráneas

¿Qué ocurre al eliminar (1, 2) en **R**?

S	A	C	R	A	B
	1	3		1	2
	2	2		2	3

Tenemos las siguientes opciones:

- No permitir eliminación
- Propagar la eliminación y también borrar (1,3) de S
- Mantener la tupla en **S** pero dejar en la llave foránea el valor en null.

Eliminar con llaves foráneas

¿Qué ocurre al eliminar (1, 2) en **R**?

S	A	C	R	A	B
	1	3		1	2
	2	2		2	3

Opción 1: no permitir la eliminación. **Default en SQL!**

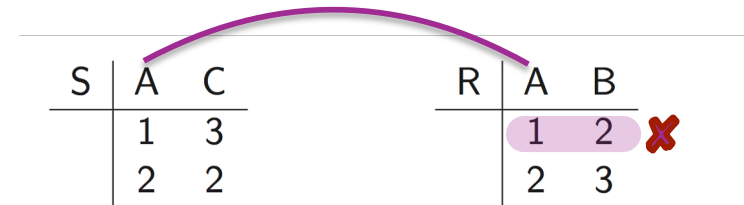
```
CREATE TABLE R(a int, b int, PRIMARY KEY(a))
```

```
CREATE TABLE S(a int, c int, FOREIGN KEY(a) REFERENCES R, ...)
```

Respuesta: obtenemos error

Eliminar con llaves foráneas

¿Qué ocurre al eliminar (1, 2) en **R**?



S	A	C	R	A	B
	1	3		1	2
	2	2		2	3

Opción 2: Propagar la eliminación. (**Cascada de eliminaciones**)

```
CREATE TABLE R(a int, b int, PRIMARY KEY(a))
```

```
CREATE TABLE S(a int, c int,  
FOREIGN KEY(a) REFERENCES R ON DELETE CASCADE, ...)
```

Respuesta: se elimina también (1, 3) en S

Eliminar con llaves foráneas

¿Qué ocurre al eliminar (1, 2) en **R**?

S	A	C
	1	3
	2	2

R	A	B
	1	2
	2	3

Opción 3: **Dejar en nulo**

```
CREATE TABLE R(a int, b int, PRIMARY KEY(a))
```

```
CREATE TABLE S(a int, c int,
```

```
  FOREIGN KEY(a) REFERENCES R ON DELETE SET NULL, ...)
```

Respuesta: la tupla (1, 3) en S ahora es (null, 3)

Producto cruz

Si pedimos datos de más de una tabla la base de datos va hacer un producto cruz y entregará **NxM** filas.

SELECT *

FROM Peliculas, Actuo_en

id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Joins

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

Podemos hacer un **JOIN** agregando un **WHERE**

Por ejemplo, para obtener todas las películas junto a los ids de los actores que participaron en ella:

SELECT *

FROM Peliculas, Actuo_en

WHERE id = id_pelicula

Observación: id es atributo de Peliculas, mientras que id_pelicula es atributo de Actuo_en

Joins

Desambiguando atributos

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

Entregue todas las películas junto a los id de los actores que participaron en ella:

SELECT *

FROM Peliculas, Actuo_en

WHERE Peliculas.id = Actuo_en.id_pelicula

Sirve cuando tenemos atributos en distintas tablas con el mismo nombre y para agregarle claridad a la consulta.

¿Y si queremos los nombres de los actores en vez de los ids?

id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

id	nombre	edad
1	Leonardo DiCaprio	41
2	Matthew McConaughey	46
3	Daniel Radcliffe	27
4	Jessica Chastain	39

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

```
SELECT Peliculas.nombre, Actores.nombre  
FROM Peliculas, Actuo_en, Actores  
WHERE Peliculas.id = Actuo_en.id_pelicula  
AND Actores.id = Actuo_en.id_actor
```

Alias

id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

id	nombre	edad
1	Leonardo DiCaprio	41
2	Matthew McConaughey	46
3	Daniel Radcliffe	27
4	Jessica Chastain	39

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

Podemos acortar la consulta anterior:

```
SELECT p.nombre, a.nombre  
FROM Peliculas AS p, Actuo_en AS ae, Actores AS a  
WHERE p.id = ae.id_pelicula  
AND a.id = ae.id_actor
```

Ese tipo de alias no es muy recomendable

Alias

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Podemos hacer operaciones y nombrar la columna:

```
SELECT (nombre || ' dirigida por ' || director) as credits, año  
FROM Peliculas
```

credits	año
Interstellar dirigida por C.Nolan	2014

Ordenando

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Entregue el nombre y la calificación de todas las películas (orden ascendente):

SELECT nombre, calificacion

FROM Peliculas

ORDER BY nombre, calificacion

El i-ésimo atributo del ORDER BY resuelve un empate en el atributo i-1

ORDER BY nombre **DESC**, calificacion

Operadores de conjuntos

- **EXCEPT**: diferencia del álgebra
- **UNION**: unión del álgebra
- **INTERSECT**: intersección del álgebra
- **UNION ALL**: unión que admite duplicados

Union

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

id	nombre	edad
1	Leonardo DiCaprio	41
2	Matthew McConaughey	46
3	Daniel Radcliffe	27
4	Jessica Chastain	39

Entregue el nombre de todos actores y directores:

SELECT nombre

FROM Actores

UNION

SELECT director

FROM Peliculas

Matching de patrones con LIKE

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

s LIKE p: string “s es como” p, donde p es un patrón

definido mediante:

- **%** Cualquier secuencia de caracteres
- **_** Cualquier caracter (solamente uno)

SELECT *

FROM Peliculas

WHERE name LIKE '%Potter%'

Conjuntos

LIKE

DISTINCT

Agregación

Anidación

Nulos

Joins

Redundancia

Eliminando
duplicados

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Entregue todos los nombres distintos de las películas:

```
SELECT DISTINCT nombre  
FROM Peliculas
```

OJO: **DISTINCT** es un operador en sí mismo.

Agregación

¿Qué hace esta consulta?

```
SELECT AVG(precio)  
FROM Productos  
WHERE fabricante = 'Toyota'
```

Entrega el precio promedio de los productos marca Toyota

También podemos usar SUM, MIN, MAX, COUNT,
etc.

Agregación

¿Qué diferencia estas consultas?

```
SELECT COUNT(*)  
FROM Productos  
WHERE año > 2012
```

```
SELECT COUNT(fabricante)  
FROM Productos  
WHERE año > 2012
```

COUNT(*) cuenta los nulos.

Ojo: En ambas se cuentan los duplicados

Ejemplo

Compra(producto, fecha, precio, cantidad)

producto	fecha	precio	cantidad
tomates	07/02	100	6
tomates	06/07	150	
zapallo	08/02	800	1
zapallo	09/07	1000	
zapallo	01/01	600	1

```
SELECT COUNT(*)  
FROM Compra
```

5

```
SELECT COUNT(cantidad)  
FROM Compra
```

3

Ejemplo

Compra(producto, fecha, precio, cantidad)

producto	fecha	precio	cantidad
tomates	07/02	100	6
tomates	06/07	150	
zapallo	08/02	800	1
zapallo	09/07	1000	
zapallo	01/01	600	1

SELECT DISTINCT COUNT(cantidad)
FROM Compra

3

SELECT COUNT(DISTINCT cantidad)
FROM Compra

2

GROUP BY

```
SELECT fabricante, COUNT(fabricante)
FROM Productos
WHERE año > 2012
GROUP BY fabricante
```

Esta consulta:

- Computa los resultados según el **FROM** y **WHERE**
- Agrupa los resultados según los atributos del **GROUP BY**
- Para cada grupo se aplica independientemente la agregación

Conjuntos

LIKE

DISTINCT

Agregación

Anidación

Nulos

Joins

Redundancia

Ejemplo

Compra(producto, fecha, precio, cantidad)

producto	fecha	precio	cantidad
tomates	07/02	100	6
tomates	06/07	150	4
zapallo	08/02	800	1
zapallo	09/07	1000	2
zapallo	01/01	600	3

SELECT producto, SUM(precio*cantidad) as ventaTotal

FROM Compra

WHERE fecha > '10/01'

GROUP BY producto

1) Se computa el FROM y el WHERE

2) Agrupar según el GROUP BY

producto	fecha	precio	cantidad
tomates	07/02	100	6
	06/07	150	4
zapallo	08/02	800	1
	09/07	1000	2

3) Agregar por grupo y ejecutar la proyección

producto	ventaTotal
tomates	1200
zapallo	2800

HAVING

Misma consulta, pero sólo queremos los productos que se vendieron más de 100 veces

```
SELECT producto, SUM(precio*cantidad) AS ventaTotal  
FROM Compra  
WHERE fecha > '10/01'  
GROUP BY producto  
HAVING SUM(cantidad) > 100
```

¿Por qué usamos HAVING y no lo incluimos en el WHERE?

Consultas con Agregación

SELECT <S>

FROM R1, ..., Rn

WHERE <Condición 1>

GROUP BY a1, ..., ak

HAVING <Condición 2>

- S puede contener atributos de $a_1, \dots, a_k$ y/o agregados, pero ningún otro atributo
- Condición 1 es una condición que usa atributos de $R_1, \dots, R_n$
- Condición 2 es una condición de agregación de los atributos de $R_1, \dots, R_n$

Evaluación

SELECT <S>

FROM $R_1, \dots, R_n$

WHERE <Condición 1>

GROUP BY $a_1, \dots, a_k$

HAVING <Condición 2>

- Se computa el FROM - WHERE de $R_1, \dots, R_n$
- Agrupar la tabla por los atributos de $a_1, \dots, a_k$
- Computar los agregados de la Condición 2 y mantener grupos que satisfacen
- Computar agregados de S y entregar el resultado

Consultas Anidadas

- Como ya habíamos visto con las operaciones de conjuntos, una consulta puede estar constituida por operaciones entre consultas.
- Pero esa no es la única forma, SQL nos ofrece mucho más.

Consultas Anidadas

Consideremos este esquema:

Bandas(nombre, vocalista, ...)

Estudiantes_UC(nombre, ...)

Toco_en(nombre_banda, nombre_festival)

Obtengamos todas las bandas cuyo vocalista sea un estudiante UC y que hayan tocado en Lollapalooza

```
SELECT Bandas.nombre
FROM Bandas, Estudiantes_UC
WHERE Bandas.vocalista = Estudiantes_UC.nombre
AND Bandas.nombre IN (
    SELECT Toco_en.nombre_banda
    FROM Toco_en
    WHERE Toco_en.nombre_festival = 'Lollapalooza'
)
```

Consultas Anidadas

Si la sub consulta retorna un escalar podemos usar los operadores condicionales típicos. Si no podemos hacerlo agregando un operador adicional.

- $s \text{ IN } R$
- $s > \text{ALL } R$
- $s > \text{ANY } R$
- $\text{EXISTS } R$

Consultas Anidadas

ALL, ANY

Cervezas(nombre, precio, ...)

Cervezas más baratas que la Austral Calafate

SELECT Cervezas.nombre

FROM Cervezas

WHERE Cervezas.precio < ALL (

SELECT Cervezas2.precio

FROM Cervezas AS Cervezas2

WHERE Cervezas2.nombre = 'Austral Calafate'

)

Consultas Anidadas

ALL, ANY

Cervezas que no son la más cara

```
SELECT Cervezas.nombre  
FROM Cervezas  
WHERE Cervezas.precio < ANY (  
    SELECT Cervezas2.precio  
    FROM Cervezas AS Cervezas2  
)
```


Consultas Anidadas

¿Podemos expresar estas consultas con **SELECT**
- **FROM** - **WHERE**?

Hint: Las consultas SFW son **monótonas**¹. Una consulta con **ALL** no es monótona. Una consulta con **ANY** lo es.

¹**Una consulta es "monótona" si agregar más datos nunca reduce el conjunto de resultados que produce.**

Sub Consultas Relacionadas

Nombres de películas que se repiten en años diferentes

```
SELECT p.nombre  
FROM Películas AS p  
WHERE p.año <> ANY (  
    SELECT año  
    FROM Películas  
    WHERE nombre = p.nombre  
)
```

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

La sub consulta depende de la externa!

Consultas Anidadas

Como Joins

El nombre de cada actor junto con el total de películas en las que ha actuado.

id	nombre	edad
1	Leonardo DiCaprio	41
2	Matthew McConaughey	46
3	Daniel Radcliffe	27
4	Jessica Chastain	39

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

```
SELECT Actores.nombre, agg.cuenta
FROM Actores, (
    SELECT id_actor as id, COUNT(*) as cuenta
    FROM Actuo_en
    GROUP BY id_actor
) as agg
WHERE Actores.id = agg.id
```

id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Conjuntos**LIKE****DISTINCT****Agregación****Anidación****Nulos****Joins****Redundan
cia**

Consultas Anidadas como Joins

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasia	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

id	nombre	edad
1	Leonardo DiCaprio	41
2	Matthew McConaughey	46
3	Daniel Radcliffe	27
4	Jessica Chastain	39

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

El nombre de cada actor junto con el año de la primera película en la que actuó.

```
SELECT Actores.nombre, agg.año
```

```
FROM Actores, (
```

```
    SELECT Actuo_en.id_actor as id, MIN(Peliculas.año) as año
```

```
    FROM Actuo_en, Peliculas
```

```
    WHERE Actuo_en.id_pelicula = Peliculas.id
```

```
    GROUP BY id_actor
```

```
) as agg
```

```
WHERE Actores.id = agg.id
```

¿Qué pasa si queremos el nombre de la película además del año?

Información Incompleta

- En una base de datos real, muy seguido no tendremos los datos para llenar todas las columnas al agregar una fila.
- También puede ser que por la lógica del problema, que un campo esté vacío tenga una semántica relevante para la aplicación.
- Con SQL podemos modelar la falta de información mediante nulos (**NULL**).
- Los nulos en las tablas generan ciertos comportamientos extraños que es bueno tener en cuenta al trabajar con ellos. Los discutiremos en esta clase.

Información Incompleta

Películas

nombre	director	actor
Django sin cadenas	Tarantino	Di Caprio
Django sin cadenas	Tarantino	Waltz
null	Tarantino	Thurman
null	Tarantino	null
El Hobbit	Jackson	McKellen
Señor de los Anillos	null	McKellen

¿Qué significan los nulos en este caso?

Significado

En el caso anterior puede significar que no se dispone de la información, pero existe!

En general los nulos pueden significar:

- Valor existe, pero no tengo la información
- Valor no existe (si nombre = null la información no existe)
- Ni siquiera sé si el valor existe o no

Consultando con nulos

Sea la relación **R**(a, b), las consultas:

- `SELECT * FROM R`
- `SELECT * FROM R WHERE R.b = 3 OR R.b <> 3`

¿Son lo mismo?

Si **R.b** es nulo, ni **R.b = 3** o **R.b <> 3** evalúan a verdadero

Consultando con nulos

La consulta:

```
SELECT * FROM R
```

Equivale a la unión de:

- ```
SELECT * FROM R WHERE R.b = 3
```
- ```
SELECT * FROM R WHERE R.b <> 3
```
- ```
SELECT * FROM R WHERE R.b IS NULL
```

Para ver si un elemento es nulo usamos **IS NULL**

Para ver si un elemento no es nulo usamos **IS NOT NULL**

# Operaciones con nulos

Si algún argumento de una operación aritmética es nulo, el resultado es nulo

Sean las siguientes instancias de **R** y **S**:

| R | A    | S | B |
|---|------|---|---|
|   | 1    |   | 2 |
|   | null |   | 3 |

La consulta **SELECT R.A + S.B FROM R, S** retorna:

| Respuesta | $R.A + S.B$ |
|-----------|-------------|
|           | 3           |
|           | 4           |
|           | null        |
|           | null        |

# Operaciones con nulos

Sean las siguientes instancias de **R** y **S**:

| R | A    | S | B |
|---|------|---|---|
|   | 1    |   | 2 |
|   | null |   | 3 |

Tenemos que  $R.A = S.B$  vale:

- FALSE cuando  $R.A = 1$  y  $S.B = 2$
- UNKNOWN cuando  $R.A = \text{null}$  y  $S.B = 2$

# Lógica de tres valores

SQL usa lógica de tres valores:

| x       | NOT x   |
|---------|---------|
| true    | false   |
| false   | true    |
| unknown | unknown |

# Ejemplo

| R | A | S | A |
|---|---|---|---|
|   | 1 |   | 1 |
|   | 2 |   | 2 |
|   |   |   | 3 |
|   |   |   | 4 |

SELECT S.A FROM S

WHERE S.A NOT IN (SELECT R.A FROM R)

Resultado: {3, 4}

# Ejemplo

| R | A    | S | A |
|---|------|---|---|
|   | 1    |   | 1 |
|   | 2    |   | 2 |
|   | null |   | 3 |
|   |      |   | 4 |

SELECT S.A FROM S

WHERE S.A NOT IN (SELECT R.A FROM R)

Resultado: tabla vacía!

# Lógica de tres valores

El resultado puede ser contraintuitivo pero es correcto

Veamos la evaluación de `3 NOT IN (SELECT R.A FROM R)`

```
3 NOT IN {1, 2, null}
= NOT (3 IN {1, 2, null})
= NOT (3 = 1 OR 3 = 2 OR 3 = null)
= NOT (false OR false OR unknown)
= NOT (unknown)
= unknown
```

# Lógica de tres valores

Además 1 NOT IN (SELECT R.A FROM R) es FALSE (Lo mismo para 2)

Para el valor 4 aplicamos el razonamiento anterior.

Resultado: tabla vacía



# Agregación

$$\frac{A}{1}$$

null

SELECT COUNT(\*) FROM R vale 2

SELECT COUNT(R.A) FROM R vale 1

# Agregación

**SELECT SUM(R.A) FROM R** retorna 1 si  $R.A = \{1, \text{null}\}$

Para funciones de agregación:

- Ignore todos los nulos
- Compute el valor de la agregación

La única excepción es COUNT(\*)

# Impedir Nulos

Al crear una tabla o agregar una columna podemos incluir una restricción para que no permita NULLs

```
CREATE TABLE <nombre> (...
 <atributo> <tipo> NOT NULL,
...)
```

# Inner Joins

Usualmente hacemos JOINS, especificando en la sentencia **FROM** de la consulta las tablas que queremos usar y en el **WHERE** las condiciones.

```
SELECT *
```

```
FROM Peliculas, Actuo_en
```

```
WHERE id = id_pelicula
```

Pero SQL tiene una sintaxis mucho más clara para expresar un join normal: **INNER JOIN** (alias **JOIN**)

Conjuntos

LIKE

DISTINCT

Agregación

Anidación

Nulos

Joins

Redundancia

# Inner Joins

Estas 3 consultas son equivalentes:

| id | nombre                   | año  | categoría | calificación | director    |
|----|--------------------------|------|-----------|--------------|-------------|
| 1  | Interstellar             | 2014 | SciFi     | 8.6          | C. Nolan    |
| 2  | The Revenant             | 2015 | Drama     | 8.1          | A. Iñárritu |
| 3  | Harry Potter             | 2011 | Fantasia  | 8.1          | D. Yates    |
| 4  | The Theory of Everything | 2014 | Biografía | 7.7          | J. Marsh    |
| 5  | Inception                | 2010 | Adventure | 8.8          | C. Nolan    |

| id | nombre              | edad |
|----|---------------------|------|
| 1  | Leonardo DiCaprio   | 41   |
| 2  | Matthew McConaughey | 46   |
| 3  | Daniel Radcliffe    | 27   |
| 4  | Jessica Chastain    | 39   |

| id_actor | id_película |
|----------|-------------|
| 1        | 2           |
| 2        | 1           |
| 4        | 1           |
| 3        | 3           |
| 1        | 5           |

SELECT \*

FROM Peliculas, Actuo\_en

WHERE id = id\_película

SELECT \*

FROM Peliculas JOIN Actuo\_en

ON id = id\_película

SELECT \*

FROM Peliculas INNER JOIN Actuo\_en

ON id = id\_película

# Outer Joins

Consideremos estas tablas:

Estudios

| Nombre | Titulo    |
|--------|-----------|
| Warner | Argo      |
| Warner | El Origen |
| MGM    | El Hobbit |

Peliculas

| Titulo     | Ingreso |
|------------|---------|
| Argo       | 136     |
| El Origen  | 292     |
| El Artista | 44      |

Escribamos una consulta que liste los ingresos totales de cada estudio.

# Outer Joins

```
SELECT Estudio.nombre, SUM(Pelicula.ingreso)
FROM Estudio JOIN Pelicula
ON Estudio.titulo = Pelicula.titulo
GROUP BY Estudio.nombre
```

- Resultado: (Warner, 428)

¿Cuál es el problema de esta consulta?

El estudio MGM, cuyas películas no tenemos información va a desaparecer por no tener contraparte en el **JOIN**.

# Outer Joins

Lo solucionamos con un Outer Join izquierdo, que mantiene las tuplas sin pareja de la primera tabla:

```
SELECT Estudio.nombre, SUM(Pelicula.ingreso)
FROM Estudio LEFT OUTER JOIN Pelicula
ON Estudio.titulo = Pelicula.titulo
GROUP BY Estudio.nombre
```

- Resultado: (Warner, 428), (MGM, null)



# Left Outer Join

SELECT \*

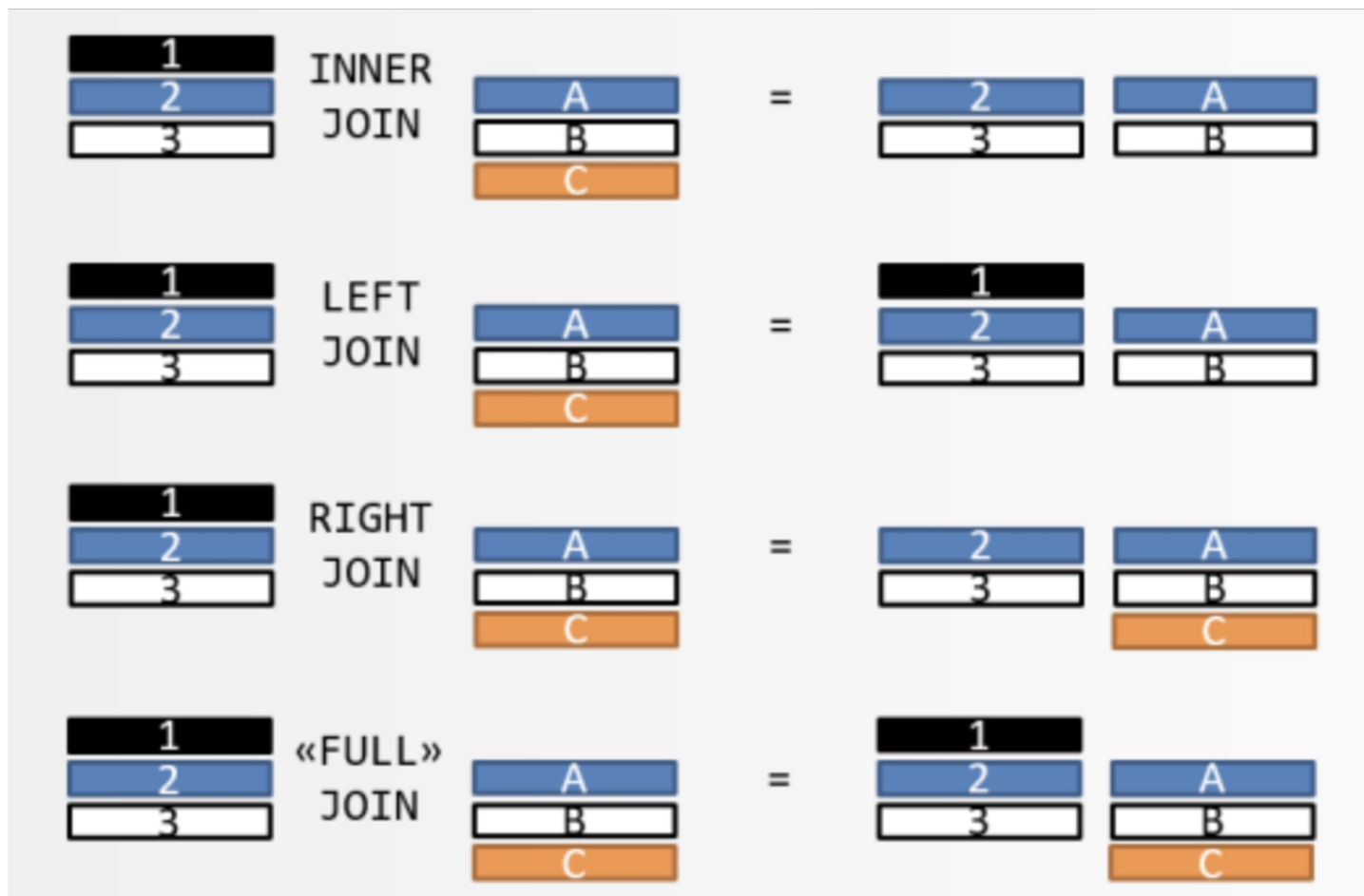
FROM Estudio LEFT OUTER JOIN Pelicula

ON Estudio.titulo = Pelicula.titulo

| nombre | titulo    | titulo    | ingreso |
|--------|-----------|-----------|---------|
| Warner | Argo      | Argo      | 136     |
| Warner | El Origen | El Origen | 292     |
| MGM    | El Hobbit | null      | null    |

# Outer Joins

- **R LEFT OUTER JOIN S**: mantenemos las tuplas de **R** que no tienen correspondencia.
- **R RIGHT OUTER JOIN S**: mantenemos las tuplas de **S** que no tienen correspondencia.
- **R FULL OUTER JOIN S**: mantenemos las tuplas de **R** y **S** que no tienen correspondencia



# Redundancia en SQL

Recordemos esta consulta:

*Bandas(nombre, vocalista, ...)*

*Estudiantes\_UC(nombre, ...)*

*Toco\_en(nombre\_banda, nombre\_festival)*

```
SELECT Bandas.nombre
```

```
FROM Bandas, Estudiantes_UC
```

```
WHERE Bandas.vocalista = Estudiantes_UC.nombre
```

```
AND Bandas.nombre IN (
```

```
 SELECT Toco_en.nombre_banda
```

```
 FROM Toco_en
```

```
 WHERE Toco_en.nombre_festival = 'Lollapalooza'
```

```
)
```

# Redundancia en SQL

¿Cómo saber cuál usar?

- Dada la naturaleza declarativa de SQL, es muy difícil predecir cómo diferentes formas de hacer una consulta puede tener un mejor rendimiento al ser ejecutadas por el RDBMS.
- Una aplicación real necesita hacer muchas consultas junto con código procedural para realizar su tarea. Cada consulta implica conectarse con la base de datos, y las conexiones tienen un *overhead* de tiempo adicional.
- Por eso en la práctica generalmente optimizamos el número de consultas a hacer, más que cómo están escritas esas consultas.

```
SELECT Bandas.nombre
FROM Bandas, Estudiantes_UC
WHERE Bandas.vocalista = Estudiantes_UC.nombre
AND Bandas.nombre IN (
 SELECT Toco_en.nombre_banda
 FROM Toco_en
 WHERE Toco_en.nombre_festival = 'Lollapalooza'
)
```

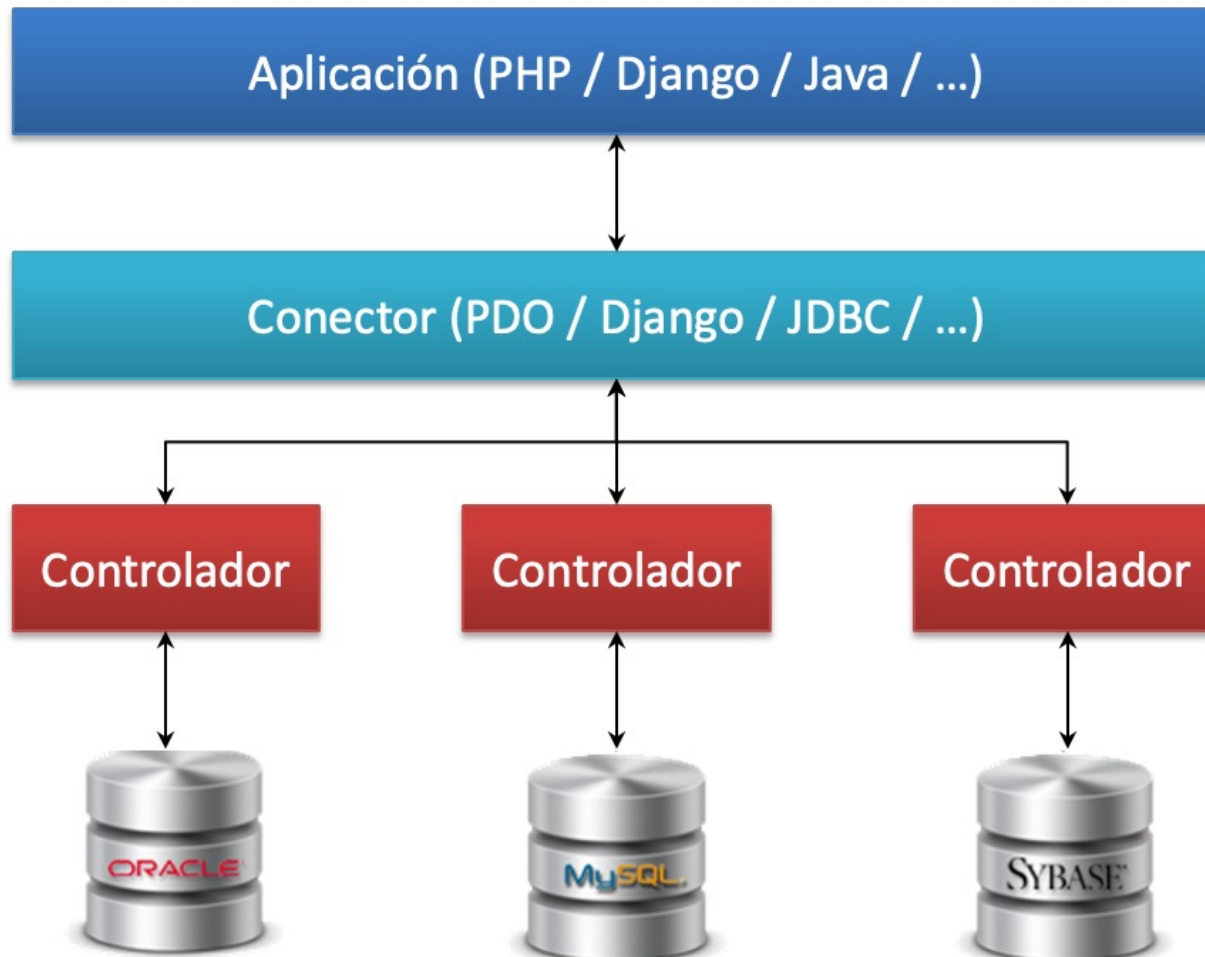
```
SELECT DISTINCT Bandas.nombre
FROM Bandas, Estudiantes_UC, Toco_en
WHERE Bandas.vocalista = Estudiantes_UC.nombre
AND Banda.nombre = Toco_en.nombre_banda
AND Toco_en.nombre_festival = 'Lollapalooza'
```

```
SELECT Bandas.nombre
FROM Bandas, Estudiantes_UC
WHERE Bandas.vocalista = Estudiantes_UC.nombre
INTERSECT
SELECT Toco_en.nombre_banda
FROM Toco_en
WHERE Toco_en.nombre_festival = 'Lollapalooza'
```

Son todas equivalentes!

# Programación y SQL

# Acceso programático





# Definir conector y consultar

- PHP /PostgreSQL

```
$conn = pg_connect("host = ... port = ... dbname = ... user = ... password = ...");
$result = pg_query($conn, "SELECT atr FROM tabla WHERE ...");
if($result){
 while ($row = pg_fetch_assoc($result)){
 echo "Result is ". $row["atr"];
 }
}
```

- PHP/PHP Data Objects (PDO)

```
$conn = new PDO("pgsql:host=...;port=...;dbname=...;user=...;password=...");
$stmt = $conn->query("SELECT atr FROM tabla WHERE ...");
while ($row = $stmt->fetch()){
 echo "Result is ". $row["atr"];
}
```

PDO ofrece más flexibilidad para cambiar el sistema de bases de datos.

# Cursores

Para ejecutar comandos SQL desde PHP, debemos usar **cursores**.

```
<?php
do {
 while ($stmt->fetch())
 ;
 if (!$stmt->nextRowset())
 break;
} while (true);
?>
```

# Cursores

Sabemos hacer consultas, ¿pero cómo le entregamos los resultados al lenguaje de programación?

**Cursores** nos entregan los resultados de una fila a la vez

```
<?php
```

```
/* Creación de un objeto PDOStatement */
```

```
$stmt = $dbh->prepare('SELECT foo FROM bar');
```

```
/* Creación de un segundo objeto PDOStatement */
```

```
$otherStmt = $dbh->prepare('SELECT foobaz FROM foobar');
```

```
/* Ejecución de la primera consulta */
```

```
$stmt->execute();
```

```
/* Recuperación de la primera fila únicamente desde el resultado */
```

```
$stmt->fetch();
```

```
/* La siguiente llamada a closeCursor() puede ser requerida por algunos drivers */
```

```
$stmt->closeCursor();
```

```
/* Ahora, podemos ejecutar la segunda consulta */
```

```
$otherStmt->execute();
```

```
?>
```

¿Por qué no nos conviene entregar todos los resultados de una sola vez?

# Casos de uso mas relevantes

- Tengo un servicio Web
- Usuarios ingresan datos
- Basado en los datos que ingresan realizo una consulta a mi base de datos, y retorno algo

# SQL Parametrizado

- Se trata de preparar una consulta con variables
- Cuando conozco el valor de las variables se instancia la nueva consulta concreta
- A su debido tiempo la ejecuto

# ¿Por que no concatenar?

<http://xkcd.com/327/>

```
query = "SELECT * FROM Students WHERE name = " + nombre

cursor.execute(query)
```

